

Flood Probability Prediction using Data Mining

Team Name: Minercrafted Insights

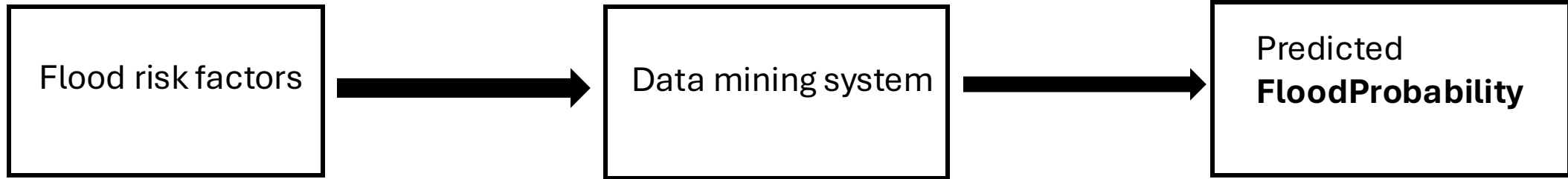
Team Members: Ashish Nadadur Chakravarthi, Akshith Reddy
Ganta, Kamal Teja Annamdasu, Ashin Kunjumon

Project Pitch Recap

Problem Statement:

The goal of this project is to:

- Study how multiple factors contribute to **Flood Probability**.
- **Identify critical drivers** among 21 human + environmental features.
- **Build predictive models** to forecast flood risk.



About the dataset:

We are using “Flood Prediction Dataset” which is provided by Kaggle.

Source: <https://www.kaggle.com/competitions/playground-series-s4e5/data>

Data Exploration

- **Dataset:** 1,117,957 rows, 22 columns, all numeric, no missing values.
- Target (**Flood Probability**): It is centered around ~0.5–0.6, indicating a **balanced dataset** for regression/classification modeling.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1117957 entries, 0 to 1117956
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype  
---  -
0   id                                     1117957 non-null  int64  
1   MonsoonIntensity                     1117957 non-null  int64  
2   TopographyDrainage                   1117957 non-null  int64  
3   RiverManagement                     1117957 non-null  int64  
4   Deforestation                        1117957 non-null  int64  
5   Urbanization                        1117957 non-null  int64  
6   ClimateChange                       1117957 non-null  int64  
7   DamsQuality                         1117957 non-null  int64  
8   Siltation                           1117957 non-null  int64  
9   AgriculturalPractices                1117957 non-null  int64  
10  Encroachments                       1117957 non-null  int64  
11  IneffectiveDisasterPreparedness      1117957 non-null  int64  
12  DrainageSystems                     1117957 non-null  int64  
13  CoastalVulnerability                 1117957 non-null  int64  
14  Landslides                          1117957 non-null  int64  
15  Watersheds                          1117957 non-null  int64  
16  DeterioratingInfrastructure          1117957 non-null  int64  
17  PopulationScore                     1117957 non-null  int64  
18  WetlandLoss                         1117957 non-null  int64  
19  InadequatePlanning                  1117957 non-null  int64  
20  PoliticalFactors                    1117957 non-null  int64  
21  FloodProbability                     1117957 non-null  float64
```

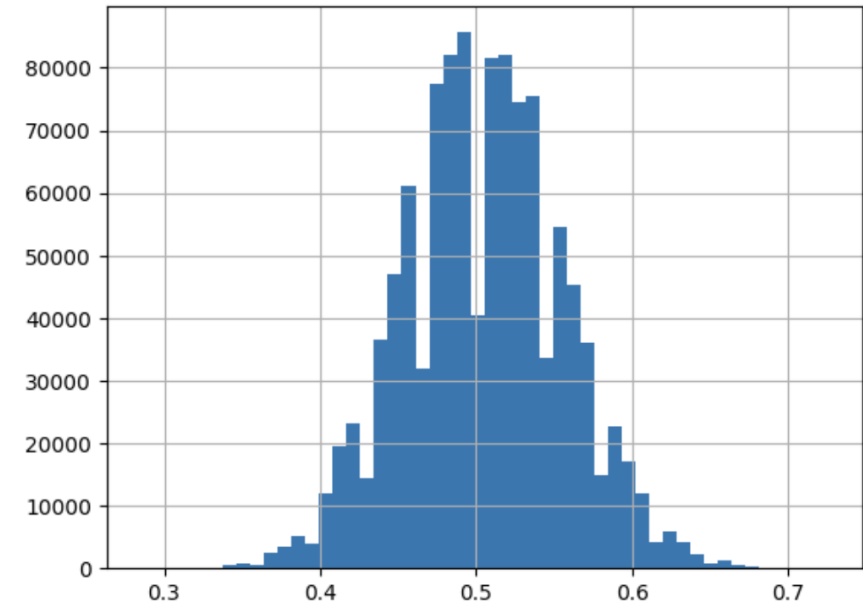
```
df.isnull().sum()
```

	0
id	0
MonsoonIntensity	0
TopographyDrainage	0
RiverManagement	0
Deforestation	0
Urbanization	0
ClimateChange	0
DamsQuality	0
Siltation	0
AgriculturalPractices	0
Encroachments	0
IneffectiveDisasterPreparedness	0
DrainageSystems	0
CoastalVulnerability	0
Landslides	0
Watersheds	0
DeterioratingInfrastructure	0
PopulationScore	0
WetlandLoss	0
InadequatePlanning	0
PoliticalFactors	0
FloodProbability	0

dtype: int64

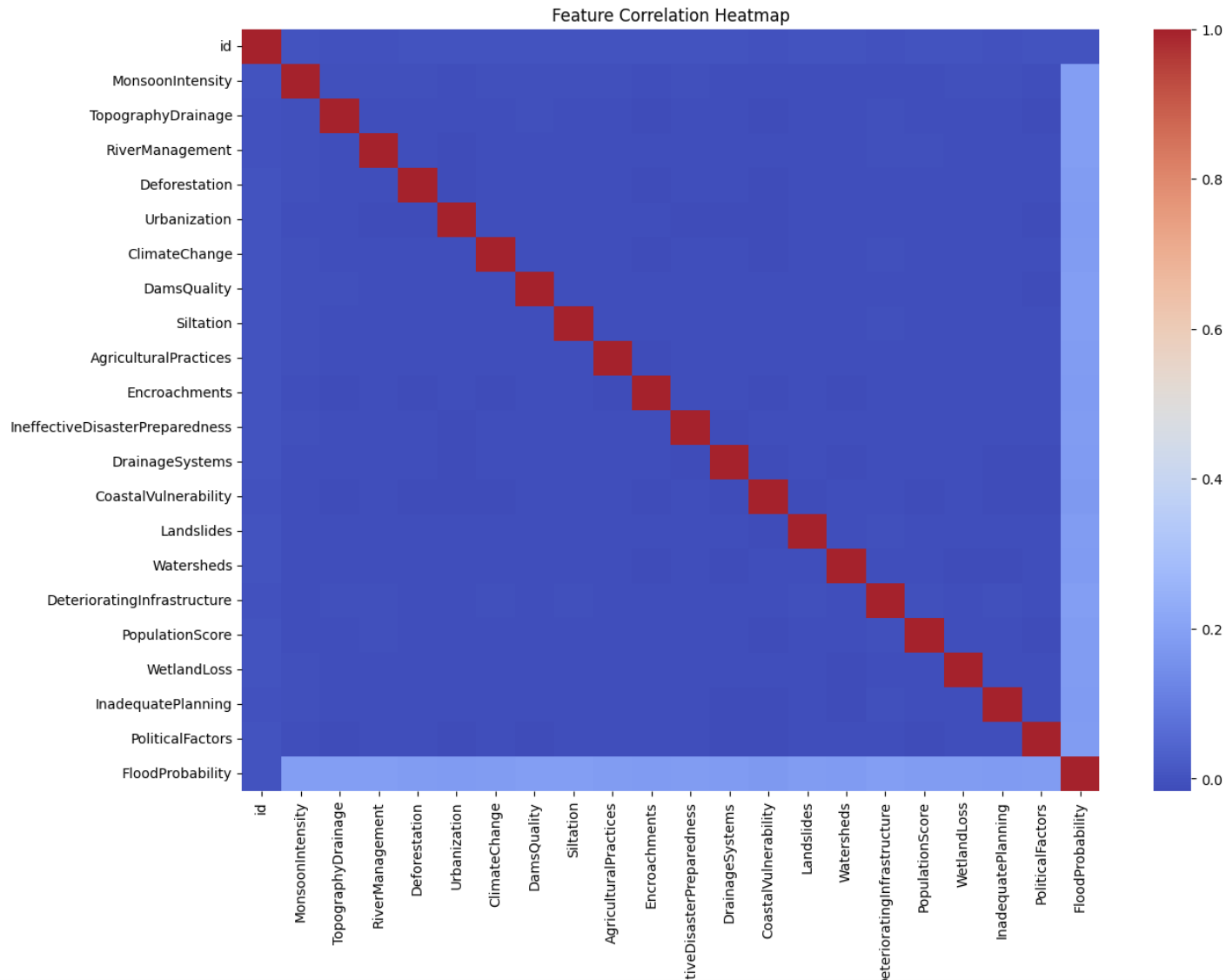
```
[22] df['FloodProbability'].hist(bins=50)
```

<Axes: >



Correlation Heatmap and Correlation Ranking with Target

```
plt.figure(figsize=(14,10))
sns.heatmap(df.corr(), cmap='coolwarm', annot=False)
plt.title('Feature Correlation Heatmap')
plt.show()
```



```
df.corr()['FloodProbability'].sort_values(ascending=False)
```

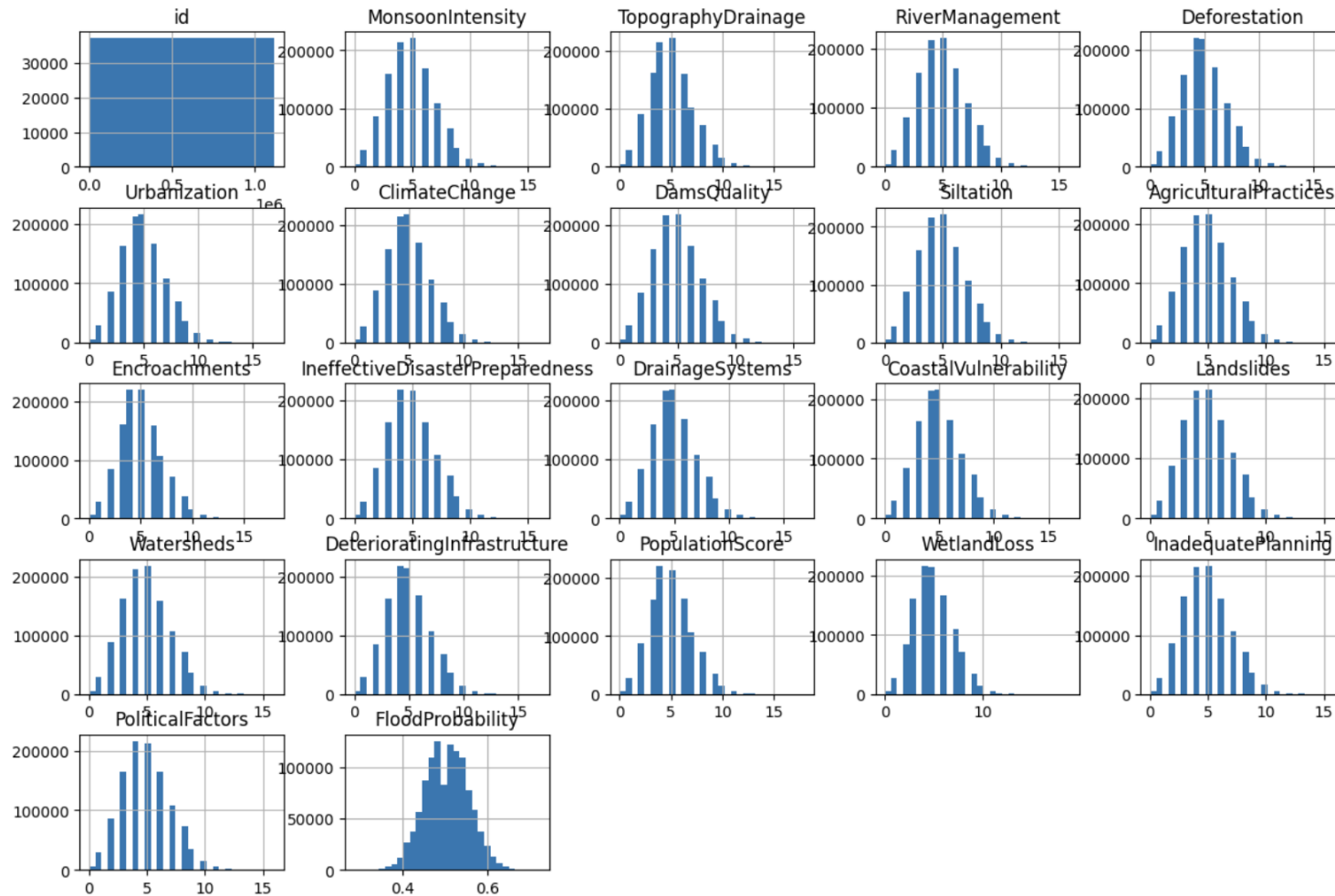
FloodProbability	
FloodProbability	1.000000
DeterioratingInfrastructure	0.190007
MonsoonIntensity	0.189098
DamsQuality	0.187996
TopographyDrainage	0.187635
RiverManagement	0.187131
Siltation	0.186789
PopulationScore	0.185890
Landslides	0.185346
ClimateChange	0.184761
Deforestation	0.184001
WetlandLoss	0.183396
AgriculturalPractices	0.183366
IneffectiveDisasterPreparedness	0.183109
PoliticalFactors	0.182417
Watersheds	0.181907
InadequatePlanning	0.180968
Urbanization	0.180861
DrainageSystems	0.179305
Encroachments	0.178841
CoastalVulnerability	0.177774
id	-0.000184

dtype: float64

Distribution Histograms

```
num_cols = df.select_dtypes('number').columns
df[num_cols].hist(bins=30, figsize=(15,10))
plt.suptitle('Feature Distributions')
plt.show()
```

Feature Distributions

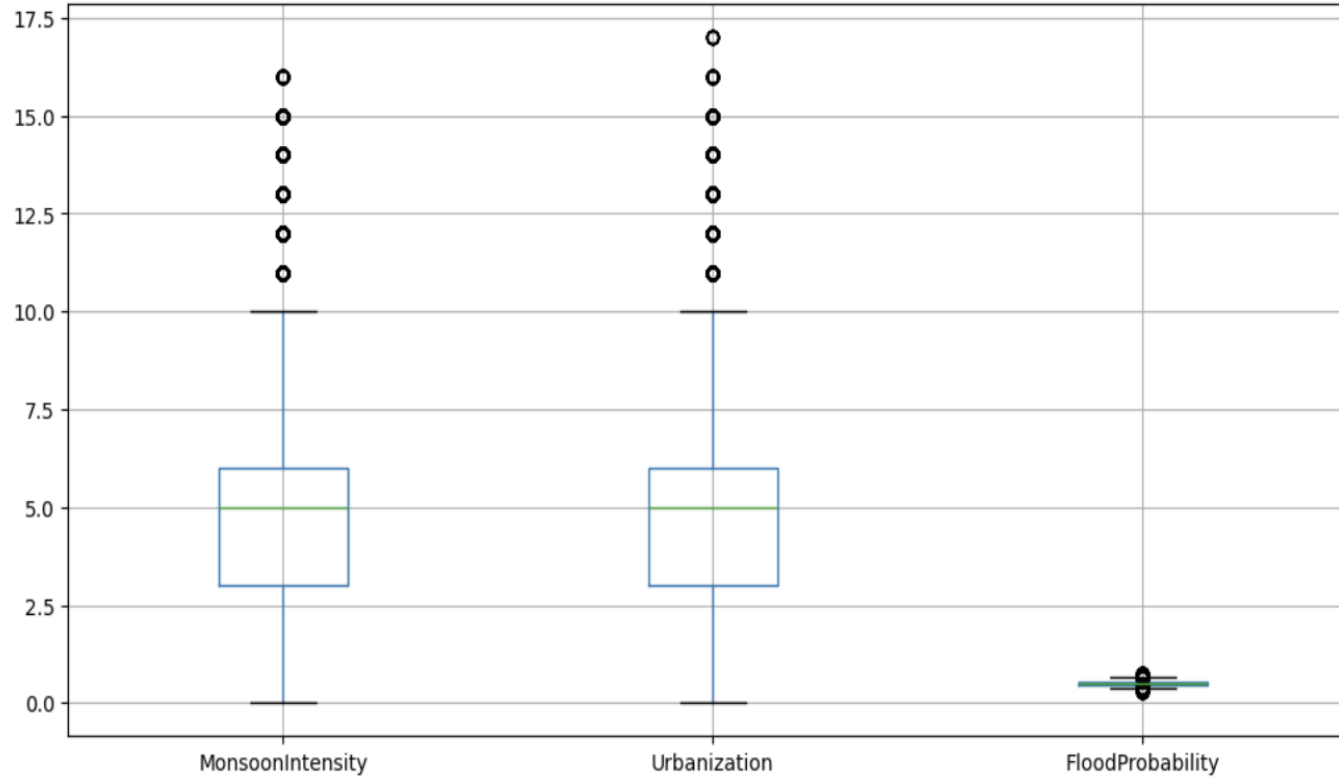


1. Most features follow roughly normal or slightly right-skewed distributions.
2. Few variables (e.g., **Urbanization**, **InadequatePlanning**) have longer tails, suggesting potential outliers or regional disparities.
3. Feature distributions are **near-normal** with mild skewness. FloodProbability is moderately centered, implying balanced samples and reliable model training potential.

Distribution Histograms

```
df.boxplot(column=['MonsoonIntensity', 'Urbanization', 'FloodProbability'], figsize=(12,6))
```

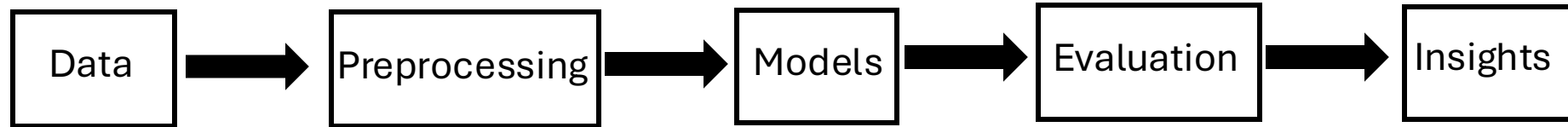
<Axes: >



1. Both MonsoonIntensity and Urbanization exhibit **visible outliers** above 10–15 (likely extreme events or rapid urban expansion zones).
2. Outliers correspond to **high-risk scenarios** and will be preserved for model training; robust modeling methods (Random Forest, XGBoost) mitigate their influence.

Data mining system pipeline

- Data exploration & preprocessing
- Model building
- Model evaluation & comparison
- Translate results into policy insights



Regression vs Classification

Regression:

1. Predict continuous flood probability (a number between 0-1)
2. **Models:** Linear Regression, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor
3. Metrics:
 - R^2 (coefficient of determination)** — how well model explains variance
 - RMSE (root mean squared error)** — magnitude of prediction error
 - MAE (mean absolute error)** — average absolute deviation

Classification:

1. By thresholding Flood Probability ≥ 0.5 to classify “**Flood**” vs “**No Flood**”.
2. **Models:** Logistic Regression, Random Forest Classifier, etc.
3. Metrics: Accuracy, Precision, Recall, F1, ROC-AUC

We explored both **classification** (predicting flood occurrence) and **regression** (predicting continuous flood probability). Since the dataset’s target variable, *FloodProbability*, is a **continuous value between 0 and 1**, framing the problem as a **regression task** better captures varying flood risk levels and provides more granular, informative predictions than a binary classification approach.

Initial Results from Regression Models

```
# Models
models = {
    "LinearRegression": LinearRegression(),
    "DecisionTree": DecisionTreeRegressor(random_state=42),
    "RandomForest": RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1),
}
```

```
) results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)

    print(f"{name}: R²={r2:.4f} | RMSE={rmse:.4f} | MAE={mae:.4f}")
    results.append({"Model": name, "R2": r2, "RMSE": rmse, "MAE": mae})

results_df = pd.DataFrame(results)
display(results_df)
```

```
LinearRegression: R²=0.8449 | RMSE=0.0201 | MAE=0.0158
DecisionTree: R²=0.0568 | RMSE=0.0495 | MAE=0.0394
RandomForest: R²=0.6584 | RMSE=0.0298 | MAE=0.0244
```

Analysis

Linear Regression ($R^2 = 0.845$)

1. 84.5 % of the variance in *Flood Probability* is explained by a linear combination of your input features.
2. Features have mostly *linear relationships* with the target — strong evidence that factors like **MonsoonIntensity**, **DeterioratingInfrastructure**, and **DrainageSystems** linearly influence flood risk.
3. **Conclusion:** Linear Regression provides a surprisingly strong baseline. It doesn't overfit the dataset easily.

Decision Tree Regressor ($R^2 = 0.057$)

1. Only ~6 % of variance is explained; the model fails to generalize.
2. Likely causes:
 1. The tree is too deep and memorizes training data (overfitting).
 2. Splitting on noisy features.
3. **Conclusion:** Decision Tree alone is unreliable for this continuous task; ensemble methods (like Random Forest) are better.

Analysis (Contd.)

Random Forest Regressor ($R^2 = 0.658$)

1. Explains 65.8 % of variance - decent, but still less than Linear Regression.
2. Default hyperparameters (e.g., limited n estimators, no depth tuning) may underfit or over-smooth predictions.
3. Ensemble improves over single Decision Tree but still underperforms Linear Regression here — which hints that:
 1. Relationships are mostly linear, and
 2. Random Forest needs parameter tuning (e.g., max_depth, min_samples_leaf) to catch subtle nonlinearities.
4. **Conclusion:** Promising nonlinear baseline; expected to outperform after tuning.

Summary and Future Plans

- 1. We trained three regression models — Linear Regression, Decision Tree, and Random Forest — to predict flood probability.
- 2. Linear Regression achieved the best fit ($R^2 = 0.845$, RMSE = 0.020), indicating dominant linear dependencies.
- 3. Tree-based models showed moderate to weak performance, suggesting the **need for tuning and advanced ensembles** in future iterations.

<u>Focus Area</u>	<u>Planned Enhancement</u>	<u>Expected Outcome</u>
Model Tuning	Optimize Random Forest (n_estimators, max_depth, min_samples_leaf)	Improve R^2 beyond 0.85
New Models	Add Gradient Boosting (XGBoost, LightGBM)	Better capture nonlinear effects
Feature Engineering	Create interaction features (e.g., Urbanization × DrainageSystems)	Reduce residual error

Future improvements will focus on **enhancing model accuracy through feature engineering and ensemble learning** (e.g., optimized Random Forest and Gradient Boosting), enabling more interpretable and adaptive flood-risk modeling.

Link to video presentation of project checkpoint 1:

https://drive.google.com/file/d/1mTLl265IViH2ZUsNQMbklCerVGPHR4_X/view?usp=sharing

THANK YOU